

Accessible Forms and Validation

Forms are conversations with users. Every label, description, error message, and focus movement is part of that conversation. Today we examine how to build forms that work for everyone — including users who navigate entirely by keyboard and screen reader.

Labels & Descriptions

Giving fields a name and context

Validation & Errors

Announcing problems clearly and precisely

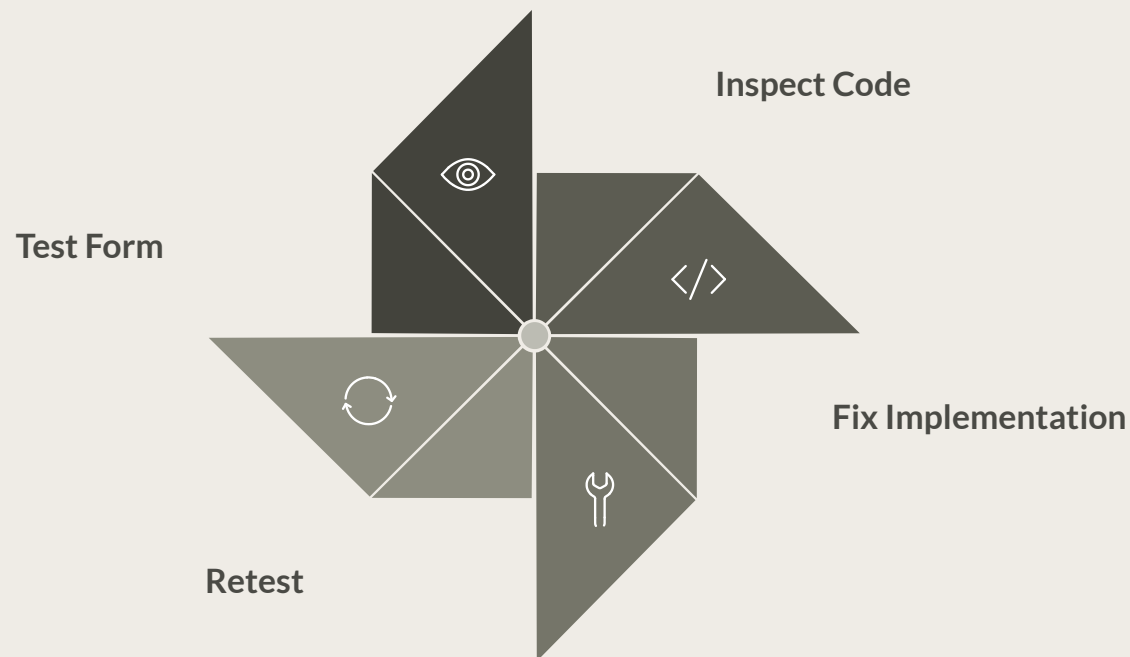
React & Angular

Preserving accessible HTML relationships in frameworks

- 📄 Speaker notes: Welcome participants to Day 2. Open 01-bad-form.html in the browser and navigate it briefly with NVDA or JAWS before saying a word. Let the screen reader speak first. Ask: "What did you notice?" This sets the tone that the rendered HTML experience — not the framework — is what matters. No files to hand out yet; today's file list is 01 through 11. Remind participants the goal is always: test first, inspect second, fix third.

Today's Learning Approach

Before reading a single line of code, we experience the form as a screen reader user would. This sequence keeps implementation grounded in real user impact rather than abstract rules.



Every exercise today follows this loop. If NVDA or JAWS does not announce what you expect, that is the bug — even if the visual design looks correct.

- Speaker notes: Emphasise that most accessibility failures are invisible on screen. A sighted developer can look at a form and think it is fine. The screen reader will tell a different story. Open 01-bad-form.html and 02-good-form.html side by side. Ask the participant: "Before I show you the code difference, can you tell me what the screen reader is missing in the bad version?" This builds diagnostic instinct, which is the core skill for the day. Files: 01-bad-form.html (bad), 02-good-form.html (good).

RECAP

Day 1 Recap

A brief grounding before we go deeper. Day 1 established the foundation that everything today builds upon.

Semantic HTML gives structure

Headings, landmarks, lists, and native elements communicate meaning to assistive technology without extra ARIA. Structure is not decoration.

Native controls give expected behaviour

Buttons, inputs, selects, and checkboxes carry keyboard interaction, focus management, and roles by default. Replacing them with `div` elements removes all of that.

ARIA should enhance, not replace

ARIA adds meaning where HTML alone cannot. It does not fix broken structure. If the underlying HTML relationship is missing, ARIA cannot rescue it.

- 📝 Speaker notes: Keep this recap tight — two minutes maximum. Ask: "Is there anything from Day 1 that felt unclear before we move forward?" If the participant raises ARIA confusion, acknowledge it and say today's form patterns will make the aria-* attribute model much more concrete. No files needed for this slide. Connect back: "Today we apply all three of these principles specifically to form relationships."

What You Will Be Able to Do by End of Day

01

Build fields with labels and descriptions

Programmatically connect every input to its visible label and any supplementary help text using `for`, `id`, and `aria-describedby`.

02

Create accessible validation and error summaries

Construct inline error messages that are announced immediately, and error summaries that move focus and list all failures after a failed submit.

03

Understand how React and Angular preserve or break form accessibility

Identify the framework-specific patterns — `htmlFor`, `bound id`, `aria-describedby` — that keep the rendered HTML accessible regardless of abstraction layer.

- 📝 Speaker notes: Read these outcomes aloud and invite the participant to flag any they already feel confident about. That shapes where you spend time. File 02-good-form.html is a working reference for outcomes 1 and 2. Files 07 and 08/09 are the React and Angular references for outcome 3. Remind the participant: "At the end of the day, exercise 10 will let you demonstrate all three outcomes at once." Ask: "Which of these three feels least familiar to you right now?"

Why Forms Fail

Form accessibility failures share a common root: the visual presentation and the programmatic structure have become disconnected. Screen readers only have access to the programmatic layer.

Text appears visually but is not connected

A placeholder or nearby span looks like a label to a sighted user. To a screen reader, the field is unlabelled and announced as "edit, blank."

Errors appear but are not announced

An error message injected into the DOM after validation may be visible on screen, yet never spoken — because it was not connected to the field or injected into a live region.

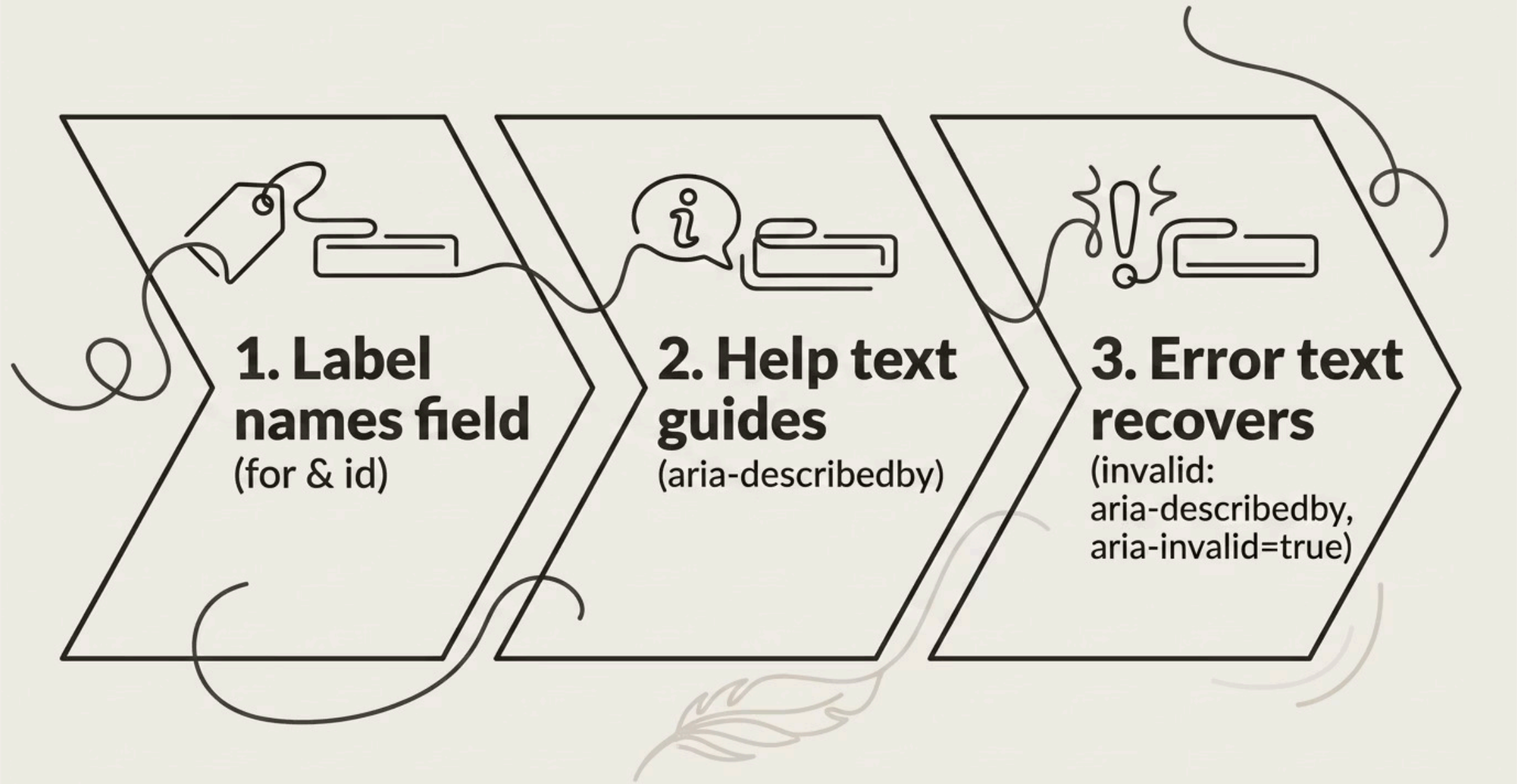
Custom components break native relationships

A styled div used as a checkbox or select removes the built-in role, keyboard behaviour, and state that assistive technology depends upon.

- ❏ Speaker notes: Open 01-bad-form.html and navigate each of these three failure types live. Use NVDA or JAWS. For the first: tab to an unlabelled field and note what is announced. For the second: submit the form and note that errors are silent. For the third: find any custom control. Ask: "Which of these three surprises you most?" Connect to code: show the missing for/id pair, the unconnected error span, and the div-based control in the HTML source. This is the diagnostic moment that motivates everything that follows.

Accessible Form Relationships

Every accessible form field is built on three programmatic relationships. Without them, the field is incomplete — regardless of how it looks visually.



These three relationships are the contract between your HTML and the screen reader. Frameworks must preserve all three in the rendered output, not just in the component source.

Speaker notes: This slide is the conceptual backbone of the entire day. Return to it after every exercise. The chevron makes clear this is not a checklist — it is a layered relationship. Ask: "If you were using JAWS and tabbed to a password field, what would you want to hear announced?" Expected answer: the label (name), the format requirements (help text), and any current error (error text). File 02-good-form.html demonstrates all three correctly. 01-bad-form.html demonstrates none.

Labels

A label gives a form field its accessible name. It is the most fundamental form relationship — and the most commonly broken one.

The correct pattern

```
<label for="email">  
  Email address  
</label>  
<input  
  type="email"  
  id="email"  
  name="email"  
>
```

What breaks it

- Using `placeholder` as the only label — it disappears on input and has poor contrast
- Using visually adjacent text in a `span` or `p` without a `for/id` connection
- Using `aria-label` when a visible label element is available — loses visual/programmatic synchronisation

- 📄 Speaker notes: Open 01-bad-form.html and tab to the first field. NVDA will announce "edit, blank" with no name. Then open 02-good-form.html and tab to the same field — the label is announced immediately. Ask: "What is the difference in your confidence as a user between those two experiences?" Explain that `for/id` is a hard link, not a heuristic. Nearby text is not sufficient. The browser exposes the label through the accessibility tree only when the programmatic relationship exists. File to reference: 02-good-form.html lines showing `for/id` pairs.

Help Text and Descriptions

`aria-describedby` allows you to attach supplementary information — format instructions, password rules, character limits — to a field without making it part of the accessible name.

Correct pattern

```
<label for="pw">Password</label>
<input
  type="password"
  id="pw"
  aria-describedby="pw-hint"
/>
<p id="pw-hint">
  8+ characters, one number,
  one symbol
</p>
```

Key principles

- The description is read after the label and role — not instead of them
- `aria-describedby` can reference multiple IDs: `aria-describedby="hint error"`
- Do not repeat the label in the description — it creates redundant announcements
- Keep descriptions concise — long descriptions become noise

📄 Speaker notes: Open 03-labels-descriptions-starter.html and navigate a password field. If help text is not connected, NVDA will not read the format rules. Then open 02-good-form.html to show the working pattern. Ask: "What would a JAWS user do if they didn't hear the password rules and submitted an invalid value?" — they would get an error with no prior guidance. That is a design failure. Explain the multiple-ID pattern for later when we add error messages to the same `aria-describedby` value. File: 03-labels-descriptions-starter.html (starter), 02-good-form.html (reference).

Exercise: Fix Labels and Descriptions

File to open

03-labels-descriptions-starter.html

Your tasks

1. Add a connected label element with matching `for` and `id` to each unlabelled field
2. Add help text paragraphs with unique IDs
3. Connect help text using `aria-describedby`
4. Test every field with NVDA or JAWS — confirm the label and description are both announced

What to listen for

- NVDA: *"Email address, edit, 8 characters minimum"* — label then description
- JAWS: announces label, type, then description on focus
- If only the label is announced, the `aria-describedby` ID does not match
- If neither is announced, the `for/id` pair is missing or mismatched



Compare your result against `02-good-form.html` to verify the full pattern.

- 📄 Speaker notes: Hand the participant `03-labels-descriptions-starter.html`. Give them 10–15 minutes. Walk around or observe screen sharing. The most common mistake is a typo in the ID — IDs are case-sensitive. If NVDA is not reading the description, ask the participant to inspect the `aria-describedby` value and compare it character for character to the element ID. Connect fix to code: show the before/after in DevTools accessibility tree. After the exercise, open `02-good-form.html` together and compare. Ask: "What changed in the accessibility tree after you added the connection?"

Grouping Controls

Radio buttons and checkboxes require a group context — the individual option labels are not enough. Users must also hear the group question.

Correct pattern

```
<fieldset>
  <legend>
    Preferred contact method
  </legend>
  <label>
    <input type="radio"
      name="contact"
      value="email">
    Email
  </label>
  <label>
    <input type="radio"
      name="contact"
      value="phone">
    Phone
  </label>
</fieldset>
```

Why this matters

Without `fieldset` and `legend`, a screen reader user hears only "Email, radio button" — with no context about what they are choosing. With the group, they hear "Preferred contact method group, Email, radio button 1 of 2."

Common mistakes

- Using a `p` or `h3` as the group label instead of `legend`
- Using `fieldset` without a `legend`
- Nesting fieldsets without clear, distinct legends

🗉 Speaker notes: Open 04-grouping-required-starter.html and navigate to a radio group without a `fieldset/legend`. Ask: "What question are you answering?" The user cannot know. Then show 02-good-form.html with proper grouping. NVDA announces the legend text as part of the group entry. JAWS behaviour is similar. Ask: "Can you think of a registration form where missing group context would cause a real problem?" — e.g. gender, payment method, subscription tier. File: 04-grouping-required-starter.html.

Required Fields

Indicating which fields are required must be done programmatically — not only with a visual asterisk or red colour.

Explain the indicator

At the top of the form, state: *"Fields marked with an asterisk (*) are required."* This must be readable text, not an image or icon alone.

Use the required attribute

Add `required` to the input element. Browsers expose this as a required state in the accessibility tree. NVDA announces "required" on focus; JAWS announces "required field."

Avoid relying on colour

Red asterisks fail colour contrast requirements for some users. The textual explanation and the `required` attribute together form the accessible pattern — colour is supplementary only.

- ❏ Speaker notes: Open 04-grouping-required-starter.html. Navigate to a field that is visually marked required but has no required attribute. NVDA will not announce "required." Add the attribute live and refresh — show the change. Ask: "Why is it problematic if a user submits a form without knowing a field is required?" — they get an error with no prior warning. File: 04-grouping-required-starter.html. Reference: 02-good-form.html for the complete required pattern including the explanatory sentence above the form.

Input Types and Autocomplete

Choosing the correct input type and declaring `autocomplete` reduces cognitive load, supports password managers, and enables mobile keyboard optimisation.



`type="email"`

Triggers email keyboard on mobile. Browsers and screen readers announce the expected format. Use with `autocomplete="email"`.



`type="tel"`

Triggers numeric keypad on mobile. Does not validate format — use pattern or server-side validation. Use with `autocomplete="tel"`.



`type="password"`

Masks input automatically. Supports password manager integration. Use `autocomplete="new-password"` for creation, `current-password` for login.



`autocomplete="name"`

Enables browser and OS autofill. Critical for users with motor disabilities who rely on autofill to avoid repetitive typing.

- ❏ Speaker notes: Open 02-good-form.html and inspect the input types and autocomplete attributes. Ask: "Why does autocomplete matter more for a screen reader user than a sighted user?" — switching between a screen reader and a keyboard to type long values is much more effortful. Avoid custom masking or JavaScript that intercepts keystrokes on these fields, as it can confuse screen readers. No separate exercise file for this slide — reinforce during exercise 1 review and the final challenge (10-final-registration-starter.html).

Masking and Live Validation

Formatting input as the user types — phone number masks, live character counts, instant validation — can create a noisy or disorienting experience for screen reader users.

What causes the problem

- DOM changes triggered on every keystroke fire `aria-live` announcements mid-thought
- Focus moving unexpectedly during masking interrupts the user's flow
- Errors shown instantly ("password too short") before the user has finished typing

Safer patterns

- Validate on `blur` (when the user leaves the field) rather than on `input`
- For forms with high stakes, validate only on submit
- If live feedback is needed, use `aria-live="polite"` — never `assertive` for format hints
- Always test live region announcements with NVDA and JAWS — they differ

- 📄 Speaker notes: Open `06-live-validation-demo.html`. Navigate to a field with live validation enabled. Tab into it, type slowly, and listen to the screen reader output. Ask: "Is this helpful or distracting?" Then switch the validation trigger to `blur` and retest. The contrast should be clear. NVDA and JAWS handle `aria-live` timing differently — NVDA tends to queue announcements while JAWS may interrupt. Always test on both. File: `06-live-validation-demo.html`.

Validation Fundamentals

When a form submission fails, every error must be specific, connected to its field, and programmatically surfaced — not just visually rendered.



Errors must be specific

"Invalid input" tells the user nothing. "Email address must include an @ symbol" gives them exactly what they need to correct the mistake.



Invalid fields need aria-invalid

Set `aria-invalid="true"` on the input when it fails validation. NVDA announces "invalid entry." JAWS announces "invalid data." Remove the attribute when the error is resolved.



Error messages must be connected

The error message element must have a unique ID, and the input's `aria-describedby` must reference it — alongside any help text already connected.

```
<input id="email" aria-invalid="true"
  aria-describedby="email-hint email-error" />
<p id="email-error" role="alert">
  Enter a valid email address</p>
```

- ❏ Speaker notes: Open 05-validation-error-summary.html. Submit the form empty. Ask: "Which errors did NVDA announce, and when?" If the `aria-invalid` and `aria-describedby` connections are not present, errors are silent. Show the code pattern on screen. JAWS and NVDA both support `role="alert"` as a live region, but timing differs. NVDA announces `role="alert"` content as soon as it is injected. JAWS may need a brief delay. File: 05-validation-error-summary.html. Reference: 02-good-form.html for the full error pattern.

Error Summaries

An error summary gives users a complete picture of what went wrong after a failed submission — before they have to navigate through each field individually.

Required pattern

```
<div
  role="alert"
  tabindex="-1"
  id="error-summary">
  <h2>
    3 errors found
  </h2>
  <ul>
    <li>
      <a href="#email">
        Enter a valid email
      </a>
    </li>
  </ul>
</div>
```

Behaviour checklist

- Move focus to the summary container using `.focus()` after failed submit
- Each error in the summary links directly to the invalid field
- The summary is injected into the DOM after validation — not pre-rendered and hidden
- NVDA and JAWS will announce `role="alert"` content automatically when injected
- Count the number of errors in the heading — sets expectation for the user

- ❏ Speaker notes: Open 05-validation-error-summary.html. Submit the form with multiple empty fields. Watch where focus goes. Ask: "Did focus move to the summary? Did JAWS read the error list?" If focus does not move, the user has no idea errors exist. Show the JavaScript that calls `.focus()` on the summary div. Connect: React and Angular must manage this focus movement explicitly — it does not happen automatically. File: 05-validation-error-summary.html.

Exercise: Inline Errors and Error Summary

File to open

05-validation-error-summary.html

Your tasks

1. Submit the form without filling in any fields
2. Listen to what NVDA or JAWS announces — note exactly what is spoken
3. Inspect the DOM: verify `aria-invalid` is set, `aria-describedby` references the error element, and `role="alert"` is present on the summary
4. If any connection is missing, add it and retest

What to listen and look for

- After submit: does focus move to the error summary container?
- Does NVDA read "3 errors found" or similar heading?
- When tabbing to an invalid field: does the screen reader announce "invalid entry"?
- Is the error message text announced as part of the field description?

- ✓ If all four are true, the pattern is complete. Activate a link in the summary — does focus land on the correct field?

📝 Speaker notes: Give the participant 15 minutes. The most common gap is focus not moving to the summary — the `.focus()` call is either missing or fires before the DOM update. Walk through the JavaScript event flow if needed. NVDA and JAWS both support `role="alert"` but JAWS sometimes needs the container to already exist in the DOM (empty) before content is injected. Ask: "How would you explain this pattern to a React developer who has never thought about focus management?" This previews the next section. File: 05-validation-error-summary.html, 02-good-form.html (reference).

Frameworks Do Not Change the Contract

The rendered HTML is what the screen reader reads.

React and Angular are tools that generate HTML. The accessibility contract — label names, descriptions, invalid states, and focus management — lives in the browser's accessibility tree, not in JSX or TypeScript templates.

Name

Every interactive element must have a programmatically determinable accessible name — whether rendered by a framework or hand-coded.

Description

Help text and error text must be connected via `aria-describedby` in the rendered HTML — not just passed as props.

State

`aria-invalid`, `aria-required`, and `aria-disabled` must be present in the rendered output and updated dynamically as state changes.

Focus

Focus management after submission, error injection, or route change must be handled explicitly. Frameworks do not do this automatically.

- ❑ Speaker notes: This is the conceptual pivot of the day — from plain HTML to frameworks. Ask: "If you inspect the rendered output of a React form in DevTools, what do you expect to see?" The answer: standard HTML. Whatever the framework produces, the browser sees HTML. Your job as a developer is to ensure that HTML is accessible. No specific file yet — this slide sets the frame for slides 18 and 19. Files 07, 08, and 09 are coming next.

React Form Component Pattern

In React, JSX syntax differs from HTML — but the rendered output must match the accessible HTML patterns established earlier today.

07-react-accessible-textfield.jsx

```
function TextField({
  id, label, hint, error
}) {
  const hintId = `${id}-hint`;
  const errorId = `${id}-error`;
  const describedBy = [
    hint ? hintId : null,
    error ? errorId : null,
  ].filter(Boolean).join( ' ');

  return (
    <div>
      <label htmlFor={id}>
        {label}
      </label>
      <input
        id={id}
        aria-describedby={describedBy}
        aria-invalid={!!error}
      />
      {hint &&
        <p id={hintId}>{hint}</p>
      }
      {error &&
        <p id={errorId}>{error}</p>
      }
    </div>
  );
}
```

Key decisions explained

- `htmlFor` is JSX's equivalent of the HTML `for` attribute — renders as `for` in the DOM
- IDs are generated from the `id` prop — avoiding duplicates across the page
- `aria-describedby` is built dynamically — only includes IDs that exist in the DOM
- `aria-invalid` is a boolean coerced to `true/false` — not a string

📝 Speaker notes: Open 07-react-accessible-textfield.jsx in the editor. Walk through the component line by line. Ask: "What happens to `aria-describedby` if there is a hint but no error?" — it should only reference the hint ID. What if there is neither? — the attribute should be omitted entirely or an empty string (which browsers ignore). Key risk: developers hard-code IDs like `id="email-hint"` — then use the component twice and create duplicate IDs. The dynamic ID pattern prevents this. Ask: "How would you test this component in the browser?" — render it, inspect the DOM, run NVDA. File: 07-react-accessible-textfield.jsx.

Angular Form Component Pattern

Angular's template syntax uses property binding to connect IDs and ARIA attributes dynamically. The goal is identical to React: correct, connected HTML in the rendered output.

08-angular-accessible-field.component.html

```
<div>
  <label [for]="fieldId">
    {{ label }}
  </label>
  <input
    [id]="fieldId"
    [attr.aria-describedby]=
      describedBy || null"
    [attr.aria-invalid]=
      hasError ? 'true' : null"
    [formControlName]="controlName"
  />
  <p *ngIf="hint"
    [id]="hintId">
    {{ hint }}
  </p>
  <p *ngIf="hasError"
    [id]="errorId">
    {{ errorMessage }}
  </p>
</div>
```

Key Angular decisions

- Use `[attr.aria-invalid]` — setting it to `null` removes the attribute from the DOM when valid
- Use `[attr.aria-describedby]` — setting to `null` when no description exists avoids empty attribute values
- Component inputs are defined in 09-angular-accessible-field.component.ts — review the `@Input()` declarations
- Always inspect rendered HTML — Angular templates are not what the screen reader reads

- 🗒 Speaker notes: Open both 08-angular-accessible-field.component.html and 09-angular-accessible-field.component.ts together. Show how the `@Input()` properties in the TypeScript file become available in the template. Ask: "What is the difference between `[aria-invalid]='true'` and `[attr.aria-invalid]='true'` in Angular?" — the first sets a property, the second sets the HTML attribute that is exposed in the accessibility tree. Only the `attr.` binding reliably produces the accessible attribute. NVDA and JAWS read attributes from the accessibility tree, not Angular's internal model. Files: 08 and 09.

Common Framework Mistakes

Framework abstraction creates distance between the code a developer writes and the HTML a screen reader reads. These are the most frequently encountered gaps.

Duplicate or unstable IDs

Reusable components that hard-code element IDs produce duplicate IDs when used more than once on a page. Screen readers and browsers associate for/id and aria-describedby with the first matching ID they encounter — breaking all subsequent instances. Always generate IDs from props or use a counter utility.

Visual labels not connected to inputs

A styled div or span acting as a label may look correct but lacks the programmatic relationship. React's htmlFor and Angular's [for] binding must reference a matching id on the input — inspect the rendered DOM to confirm, not the JSX or template.

Errors shown visually but not linked programmatically

An error injected into the DOM after validation is invisible to a screen reader user unless: the input's aria-describedby references the error element, aria-invalid="true" is set on the input, and the error text is either in a live region or focus has moved to the summary.

- ❏ Speaker notes: Ask the participant to open 07-react-accessible-textfield.jsx and deliberately break it in three ways: hard-code the hint ID, remove htmlFor, remove aria-describedby. Reload in the browser, run NVDA, and confirm what breaks. This destructive testing builds diagnostic intuition. Then restore the correct code. Ask: "Which of these three mistakes would be hardest to catch in a code review without running a screen reader?" — almost certainly the visually-linked-but-not-programmatically-linked label. Files: 07-react-accessible-textfield.jsx (primary), 08/09 (Angular equivalent).

Final Mini Challenge

File to open

10-final-registration-starter.html

Reference solution: 11-final-registration-solution.html

Time

30–45 minutes

What to build or improve

1. Ensure every field has a connected label with matching for/id
2. Add help text connected via aria-describedby
3. Mark required fields with both the required attribute and a textual explanation
4. Group radio buttons and checkboxes with fieldset and legend
5. Implement inline validation with aria-invalid and connected error messages
6. Add an error summary that receives focus after failed submission
7. **Optional extension:** Convert one field into a React or Angular component using the patterns from slides 18–19

- ❏ Speaker notes: This is the integrative exercise for the day. The participant should work through the checklist methodically, testing with NVDA or JAWS after each change — not at the end. Walk alongside them or observe. If they finish early, encourage the React or Angular extension. Ask after each section: "What does the screen reader say now? What changed?" Compare against 11-final-registration-solution.html to discuss any differences — the solution is not prescriptive, it is a reference. Files: 10-final-registration-starter.html (exercise), 11-final-registration-solution.html (solution reference).

What Makes a Form Accessible

Relationships. Not rules.

Accessible forms depend on relationships

Label to input, description to input, error to input, group question to group options.

Every relationship must be programmatic — not just visual.

Framework syntax is not the accessibility layer

JSX, TypeScript templates, and component abstractions are authoring tools. They do not create accessibility — they can either preserve or destroy the HTML relationships that do.

The rendered HTML experience is what matters

Inspect the DOM. Run the screen reader. Listen to what is — and is not — announced. That is the ground truth. Fix what you hear, not what you assume.

- 📄 Speaker notes: Close the day by returning to 01-bad-form.html and 02-good-form.html side by side. Navigate both with NVDA or JAWS one final time. Ask: "What do you hear now that you would not have noticed this morning?" The participant should be able to name specific announcements — label, description, invalid state, error summary focus. Remind them: the CPACC and WAS frameworks describe what accessibility requires; today's work is about how to implement it in production code. All files from the day are theirs to keep and use as implementation references: 01 through 11. Thank them for the day.